

PROGRAMMING 2 (PRG621S) GROUP ASSIGNMENT 2026

PROGRAMMING 2 (PRG621S) GROUP ASSIGNMENT 2026			
Project:	Java OOP & Database Application Development		
Students:	Titus Shaangelo 22208477 Shoopala shoopala 224073249 Barnabas liuhwa 223081701 Kevin Karuhumba 223085219 Ethan Paul Kennedy 224033816 Hamutenya Faith 223017183		
Qualifications:	B.COMP.SC. IN CYBER SECURITY B.INFORMATICS		
Nature of Case: (Lab Objective)	The objective of this lab was to design and implement a fully functional Rental Management System for a property company. The system was developed using Java, applying Object-Oriented Programming principles, GUI design, and database integration. The system allows users to manage properties, tenants, leases, payments, and generate reports through a centralized dashboard.		
Due Date:	Friday, 8 May 2026, 11:59 PM		
Description of Evidence	Vendor Name	Model /Serial No.	
1	Intelli J	Intelli J IDEA	version 2023.1.0
2	PostgreSQL Database (james_property_db)	PostgreSQL	PostgreSQL 16+
3	JDBC Connector Library	PostgreSQL	PostgreSQL JDBC Driver (org.postgresql.Driver)
4	Database Connection Configuration	API	jdbc:postgresql://localhost:5432/james_property_db
5	Java Source Code Files	Oracle	JDK 17/21
6	PostgreSQL Server	PostgreSQL	Localhost Server Instance

1. Introduction

The Rental Management System is a Java-based application designed to automate property rental operations for a real-world client scenario. The system replaces manual record keeping using spreadsheets and paper files with a centralized digital platform. It integrates a PostgreSQL database with a Java Swing graphical user interface to manage properties, tenants, leases, and payments. The system applies object-oriented programming concepts such as inheritance, encapsulation, and abstraction to ensure modularity and maintainability.

The main purpose of the system is to help property owners manage rental information more efficiently. The application stores all data securely in a PostgreSQL database and allows users to easily add, update, search, and manage records through a graphical interface. The system also generates reports and keeps track of payments and lease agreements.

2. Lab Procedures

Step 1: Planning the System

The first step was to analyze the project requirements and identify the main functionalities needed in the system. The project was divided into modules such as Property Management, Tenant Management, Lease Management, Payment Management, and Reporting.

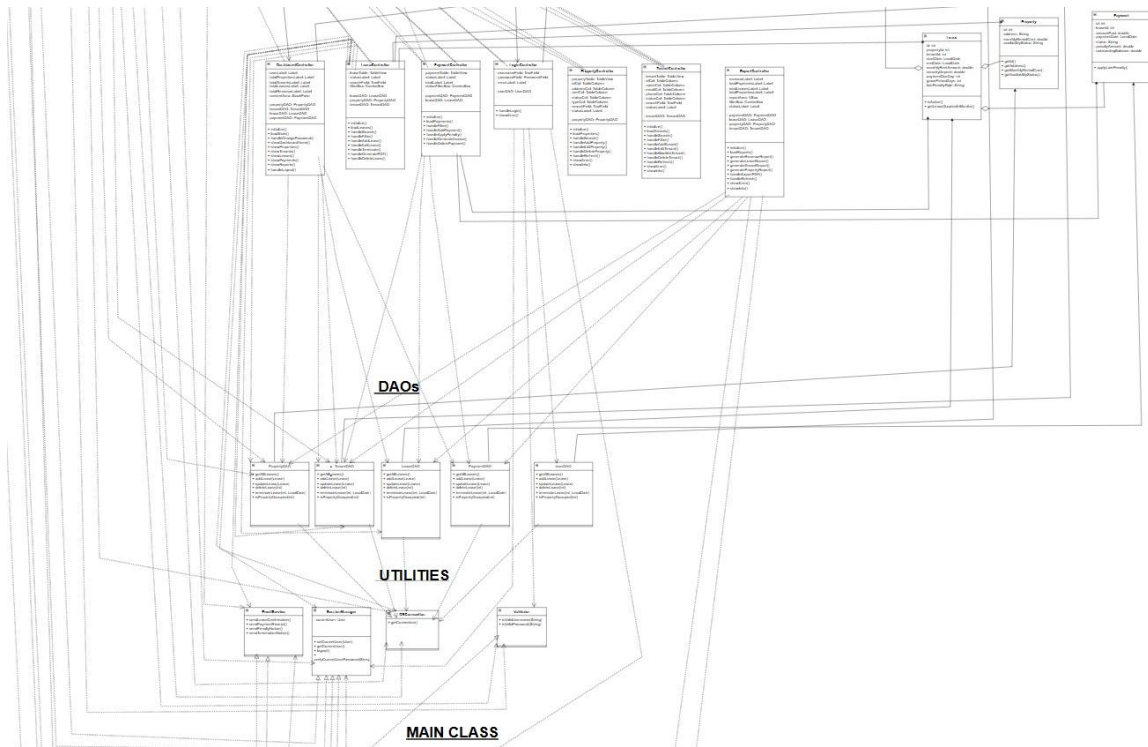
Step 2: Designing the UML Class Diagram

A UML Class Diagram was created to show the relationships between classes in the system. The diagram included inheritance, aggregation, and associations between classes such as Property, Tenant, Lease, and Payment.

Screenshot 1:

Screenshot of UML Class Diagram.

Explanation of Screenshot:



This screenshot shows the UML Class Diagram used to design the system structure. It demonstrates the relationships between classes and how object-oriented programming concepts were applied.

Step 3: Creating the PostgreSQL Database

A PostgreSQL database called `james_property_db` was created and configured for the system. Tables such as `users`, `properties`, `tenants`, `leases`, and `payments` were created using SQL scripts. Primary keys and foreign keys were also added to maintain relationships between tables.

Screenshot 3:

Screenshot of:

- PostgreSQL database
- Tables created
- SQL script

Explanation of Screenshot:

The screenshot shows the pgAdmin 4 interface with the following SQL queries in the Query Editor:

```

1 CREATE TABLE users (
2     id SERIAL PRIMARY KEY,
3     username VARCHAR(50) UNIQUE NOT NULL,
4     password_hash VARCHAR(255) NOT NULL,
5     role VARCHAR(20) NOT NULL
6 );
7
8 CREATE TABLE properties (
9     id SERIAL PRIMARY KEY,
10    property_type VARCHAR(20) NOT NULL,
11    building_floor_size NUMERIC(10,2) NOT NULL,
12    address VARCHAR(255) NOT NULL,
13    erf_number VARCHAR(50) NOT NULL,
14    location VARCHAR(100) NOT NULL,
15    market_value NUMERIC(12,2) NOT NULL,
16    monthly_rental_cost NUMERIC(10,2) NOT NULL,
17    availability_status VARCHAR(20) NOT NULL DEFAULT 'Available',
18    plot_size NUMERIC(10,2),
19    unit_number VARCHAR(50),
20    has_backyard BOOLEAN,
21    floor_level INT,
22    has_elevator_access BOOLEAN
23 );
24
25 CREATE TABLE tenants (
26     id SERIAL PRIMARY KEY,
27     first_name VARCHAR(100) NOT NULL,
28     last_name VARCHAR(100) NOT NULL,
29     id_or_passport_number VARCHAR(50) UNIQUE NOT NULL,
30     phone_number VARCHAR(50) NOT NULL,
31     email_address VARCHAR(150) NOT NULL,
32     status VARCHAR(20) NOT NULL DEFAULT 'Active'
33 );
    
```

Total rows: CR

```

32     status VARCHAR(20) NOT NULL DEFAULT 'Active'
33 );
34
35 CREATE TABLE leases (
36     id SERIAL PRIMARY KEY,
37     property_id INT NOT NULL REFERENCES properties(id),
38     tenant_id INT NOT NULL REFERENCES tenants(id),
39     start_date DATE NOT NULL,
40     end_date DATE NOT NULL,
41     monthly_rent_amount NUMERIC(10,2) NOT NULL,
42     security_deposit NUMERIC(10,2) NOT NULL,
43     payment_due_day INT NOT NULL,
44     grace_period_days INT NOT NULL DEFAULT 5,
45     late_penalty_rule VARCHAR(100)
46 );
47
48 -- PAYMENTS TABLE
49 CREATE TABLE payments (
50     id SERIAL PRIMARY KEY,
51     lease_id INT NOT NULL REFERENCES leases(id),
52     amount_paid NUMERIC(10,2) NOT NULL,
53     payment_date DATE NOT NULL,
54     status VARCHAR(20) NOT NULL DEFAULT 'Pending',
55     penalty_amount NUMERIC(10,2) NOT NULL DEFAULT 0,
56     outstanding_balance NUMERIC(10,2) NOT NULL DEFAULT 0
57 );
58
59 INSERT INTO users (username, password_hash, role)
60 VALUES ('admin', '$2a$10$N9qo8uL0ickgx2ZMRZoMyeIjZAgcfl7p92ldGxad68LJZdL17lhWy', 'Admin');
    
```

This screenshot shows the database structure used in the system. The tables were created to store rental management information permanently.

Step 4: Setting Up JDBC Connection

Screenshot 4:

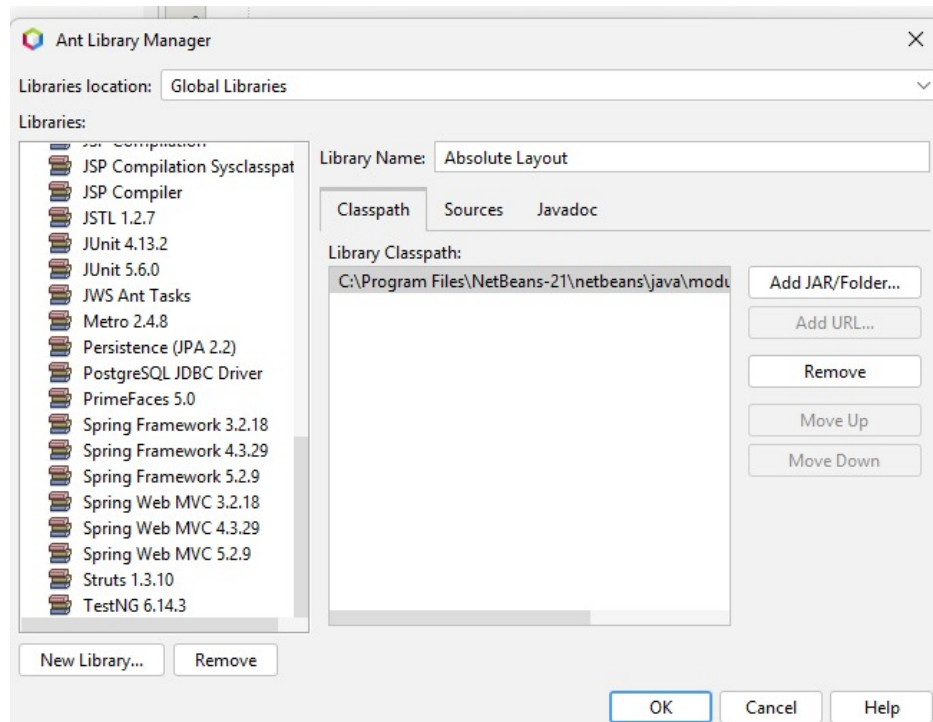
This screenshot shows the JDBC configuration used to connect the Java application to the PostgreSQL database. It includes the PostgreSQL JDBC Driver, the database connection URL, and the DBConnection.java class used to establish the connection.

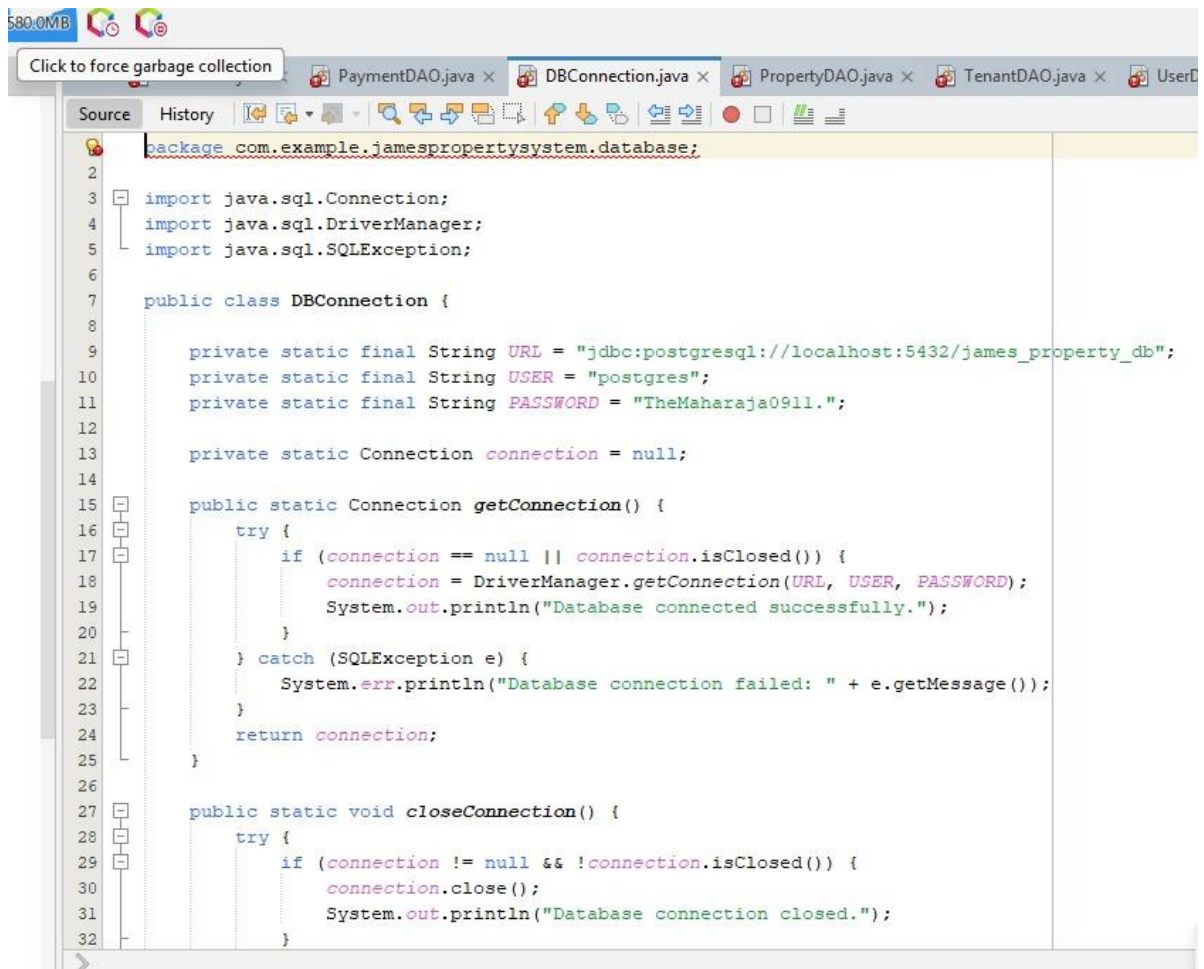
Screenshot of:

- Libraries section showing PostgreSQL Connector JAR OR
- DBConnection.java class

Explanation of Screenshot:

This screenshot shows the JDBC setup used to connect the Java application to the PostgreSQL database.





```

package com.example.jamespropertyssystem.database;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

public class DBConnection {

    private static final String URL = "jdbc:postgresql://localhost:5432/james_property_db";
    private static final String USER = "postgres";
    private static final String PASSWORD = "TheMaharaja0911.";

    private static Connection connection = null;

    public static Connection getConnection() {
        try {
            if (connection == null || connection.isClosed()) {
                connection = DriverManager.getConnection(URL, USER, PASSWORD);
                System.out.println("Database connected successfully.");
            }
        } catch (SQLException e) {
            System.err.println("Database connection failed: " + e.getMessage());
        }
        return connection;
    }

    public static void closeConnection() {
        try {
            if (connection != null && !connection.isClosed()) {
                connection.close();
                System.out.println("Database connection closed.");
            }
        }
    }
}

```

Step 5: Creating Java Classes

Different Java classes were created for the system including:

- Property
- House
- Townhouse
- Flat
- Tenant
- Lease
- Payment

Inheritance was used where House, Townhouse, and Flat inherited from the Property class.

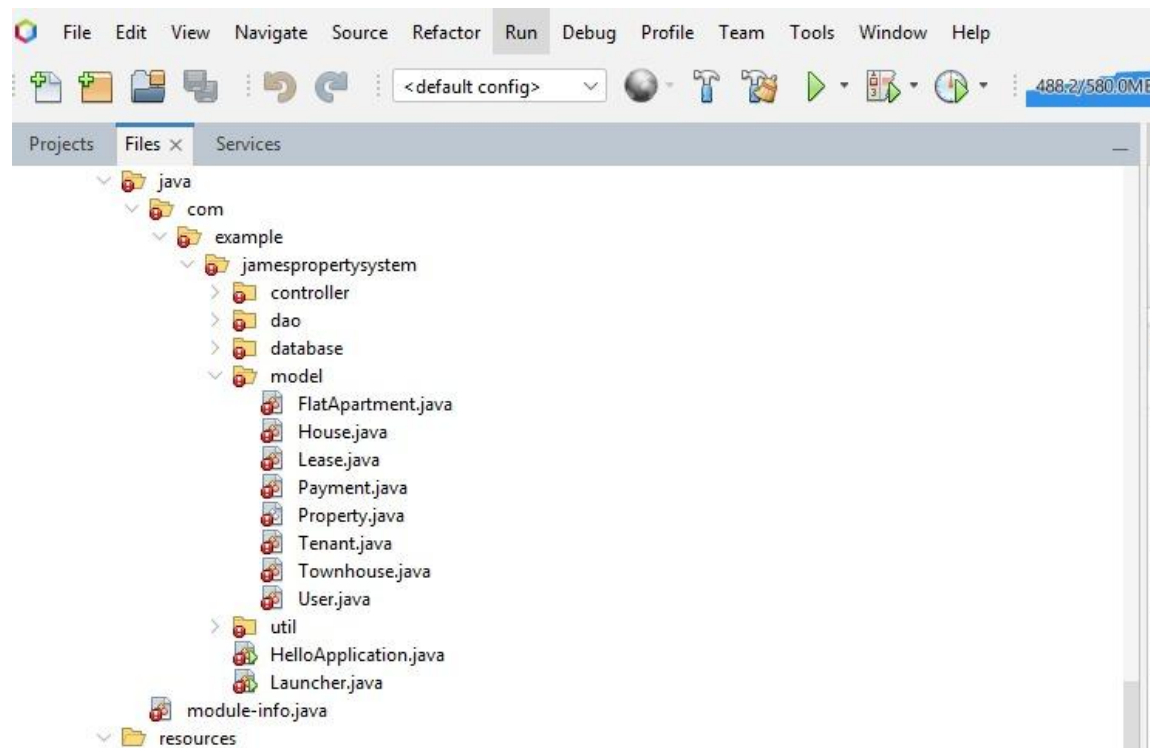
Screenshot 5:

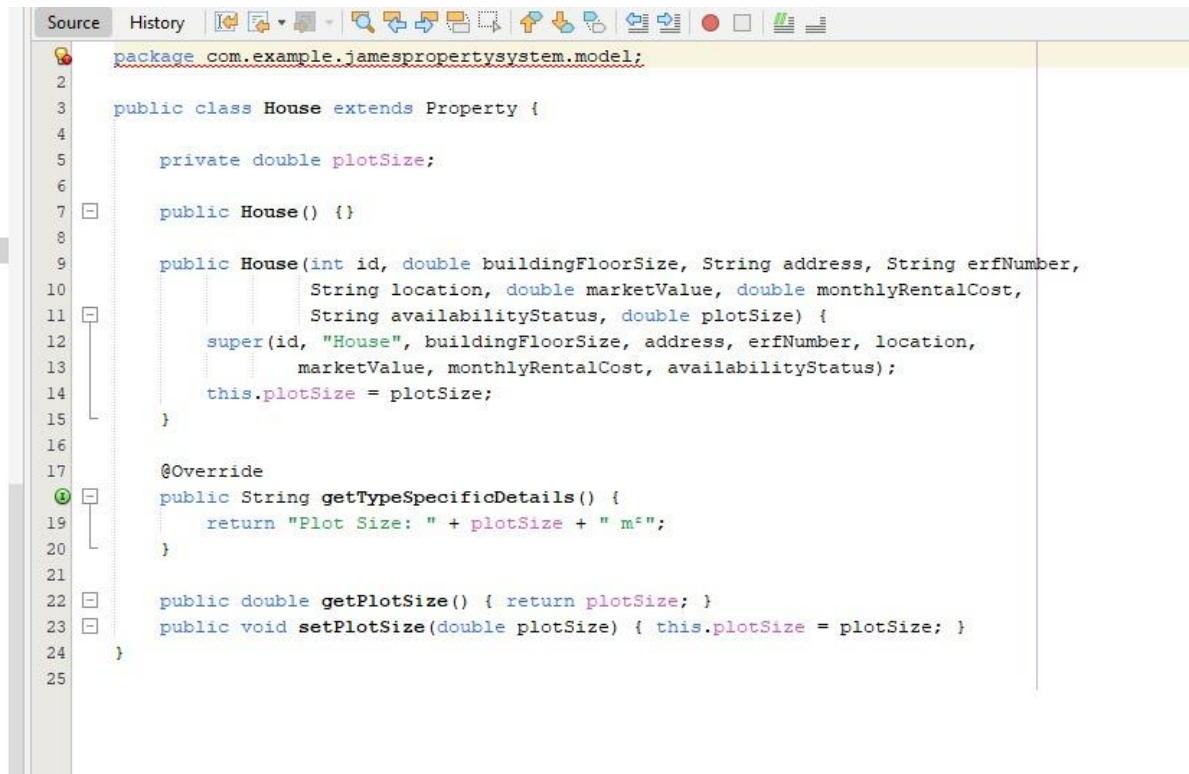
Screenshot of:

- Java classes in NetBeans/IntelliJ
- OR
- Inheritance code

Explanation of Screenshot:

This screenshot shows the Java classes created for the system and how inheritance was implemented using parent and child classes.





```

Source History
package com.example.jamespropertyssystem.model;

2
3 public class House extends Property {
4
5     private double plotSize;
6
7     public House() {}
8
9     public House(int id, double buildingFloorSize, String address, String erfNumber,
10         String location, double marketValue, double monthlyRentalCost,
11         String availabilityStatus, double plotSize) {
12         super(id, "House", buildingFloorSize, address, erfNumber, location,
13             marketValue, monthlyRentalCost, availabilityStatus);
14         this.plotSize = plotSize;
15     }
16
17     @Override
18     public String getTypeSpecificDetails() {
19         return "Plot Size: " + plotSize + " m²";
20     }
21
22     public double getPlotSize() { return plotSize; }
23     public void setPlotSize(double plotSize) { this.plotSize = plotSize; }
24 }
25

```

Step 6: Designing the Graphical User Interface (GUI)

The graphical user interface was developed using Java Swing. Forms were created for:

- Login
- Dashboard
- Add Property
- Add Tenant
- Lease Management
- Payment Management

Buttons, labels, text fields, combo boxes, and tables were used to make the application user-friendly.

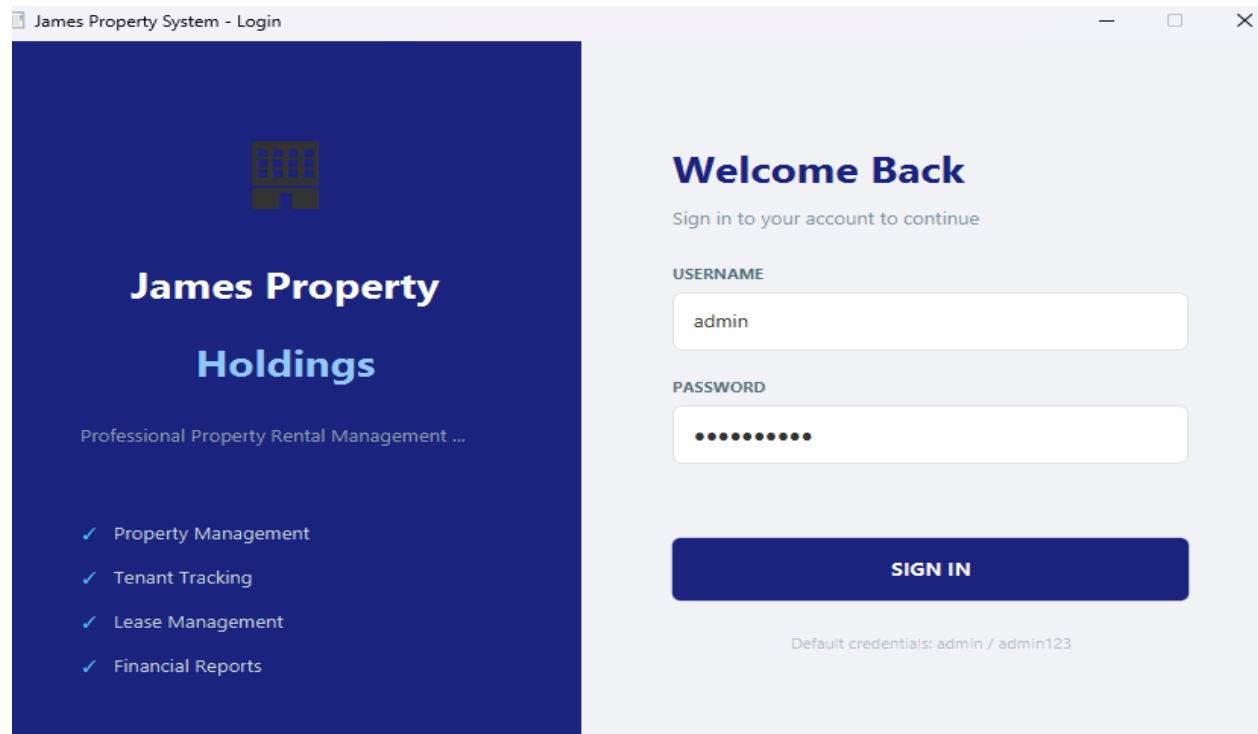
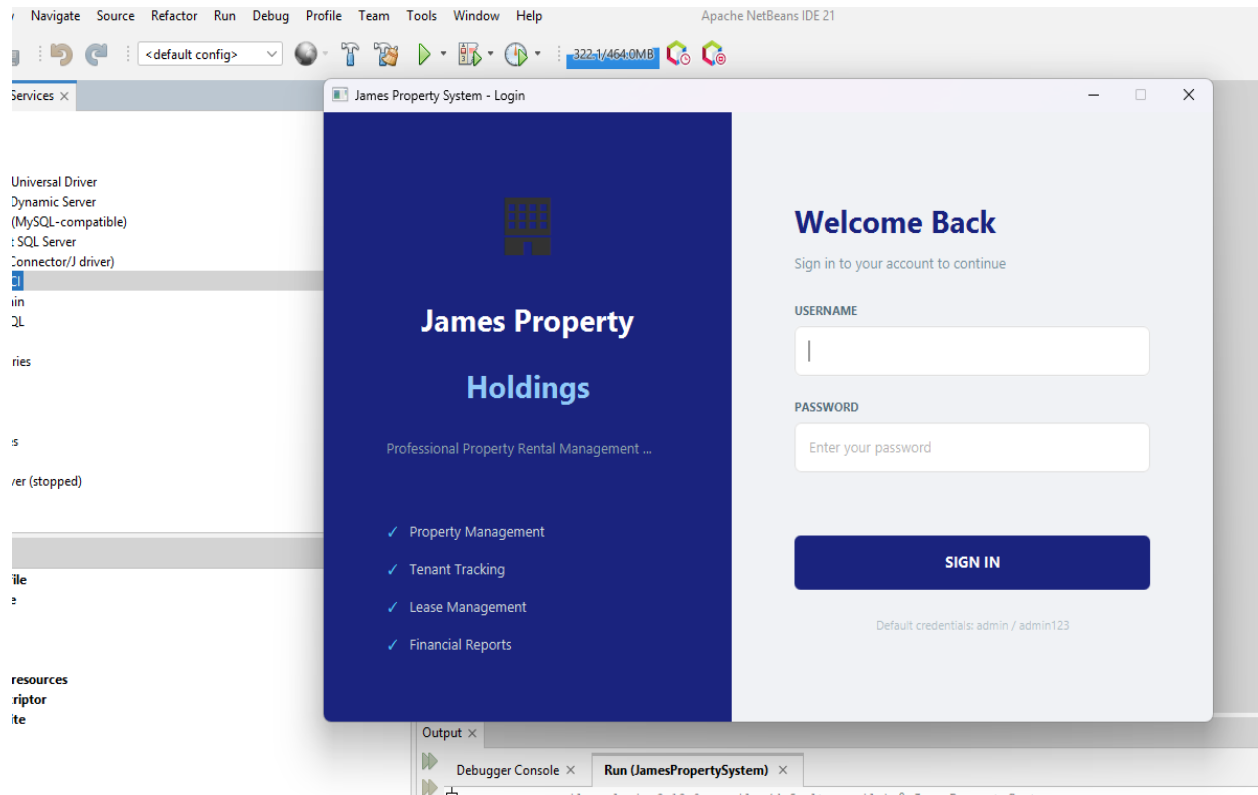
Screenshot 6:

Screenshot of Login Form.

Explanation of Screenshot:

This screenshot shows the login screen used to restrict unauthorized access to the system.

PROGRAMMING 2 (PRG621S) GROUP ASSIGNMENT 2026

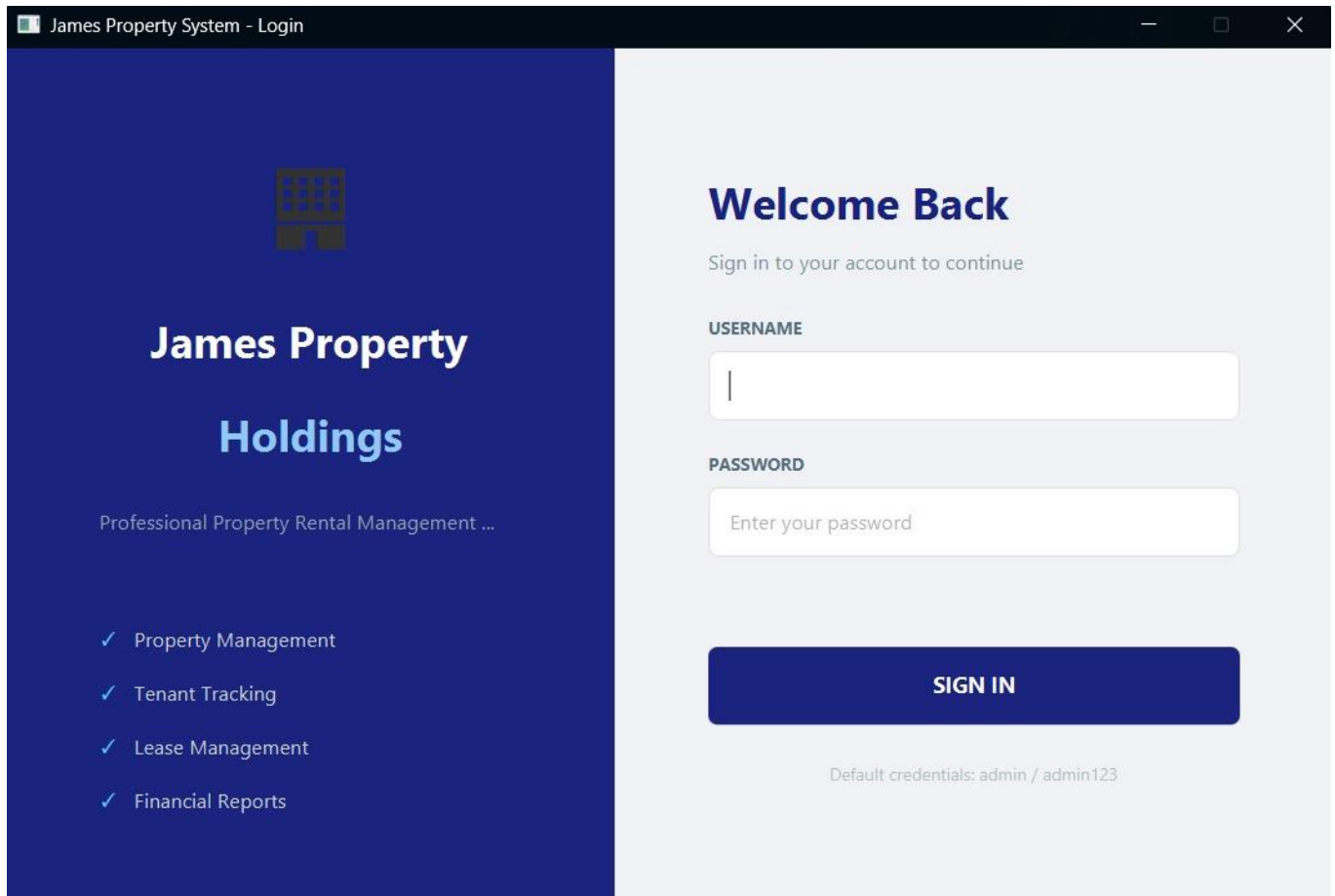


Screenshot 7:

Screenshot of Dashboard/Main Menu.

Explanation of Screenshot:

This screenshot shows the dashboard interface that allows users to navigate to different sections of the system.



James Property System - Login

James Property Holdings

Professional Property Rental Management ...

- ✓ Property Management
- ✓ Tenant Tracking
- ✓ Lease Management
- ✓ Financial Reports

Welcome Back

Sign in to your account to continue

USERNAME

PASSWORD

SIGN IN

Default credentials: admin / admin123

Screenshot 8:

Screenshot of Add Property Form.

Explanation of Screenshot:

This screenshot shows the property management form where users can register and manage rental properties.

The screenshot displays the 'James Property System' dashboard. The main header shows the user is logged in as 'admin (Admin)' with options to 'Change Password' or 'Logout'. The left sidebar contains a 'NAVIGATION' menu with links to 'Dashboard', 'Properties', 'Tenants', 'Leases', 'Payments', and 'Reports'. The 'Dashboard' link is currently selected.

The main content area is titled 'Property Management' with the subtitle 'Manage all your rental properties'. It features a 'Filter by:' dropdown set to 'All' and a table with columns 'ID', 'Type', and 'Address'. The table contains one row with ID '8', Type 'House', and Address '12 independence'. Below the table are buttons for 'Edit Selected' and 'Delete Selected', and a status indicator 'Total: 1 properties'.

An 'Add New Property' modal is open, prompting the user to 'Fill in the property details below'. The form includes the following fields:

- Property Type:
- Address*:
- ERF Number*:
- Location*:
- Floor Size (m²)*:
- Market Value (N\$)*:
- Monthly Rent (N\$)*:
- Status:
- Plot Size (House):
- Unit Number:
- Floor Level (Flat):
- ☒ Has Backyard
- ☐ Has Elevator Access

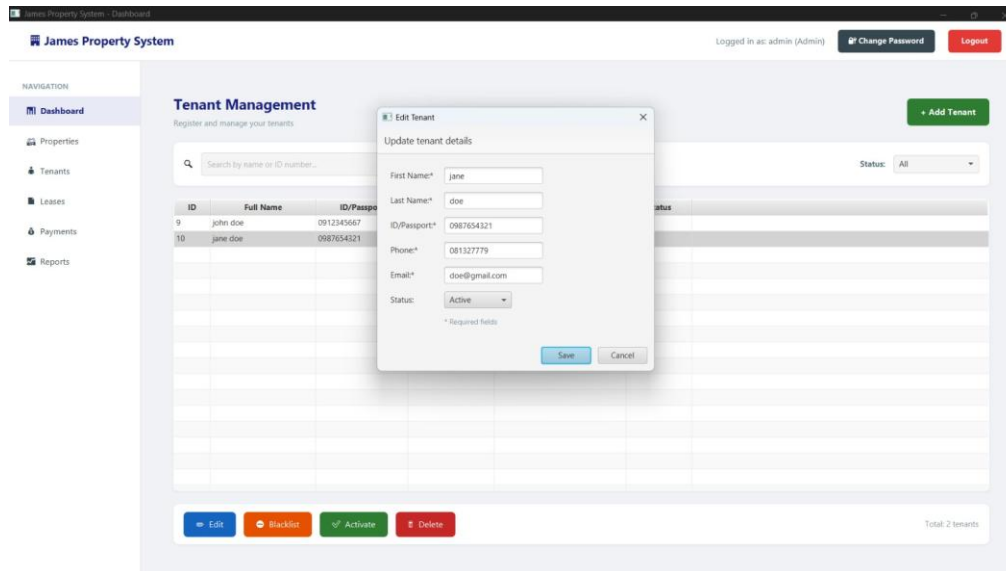
A note at the bottom of the modal states '* Required fields'. The modal has 'Save' and 'Cancel' buttons.

Screenshot 9:

Screenshot of Tenant Registration Form.

Explanation of Screenshot:

This screenshot shows the tenant registration form used to capture tenant information.



Step 7: Implementing Lease Management

The lease module was developed to connect tenants with properties. The system prevents leasing of already occupied properties and stores lease details such as start date, end date, and monthly rental amount.

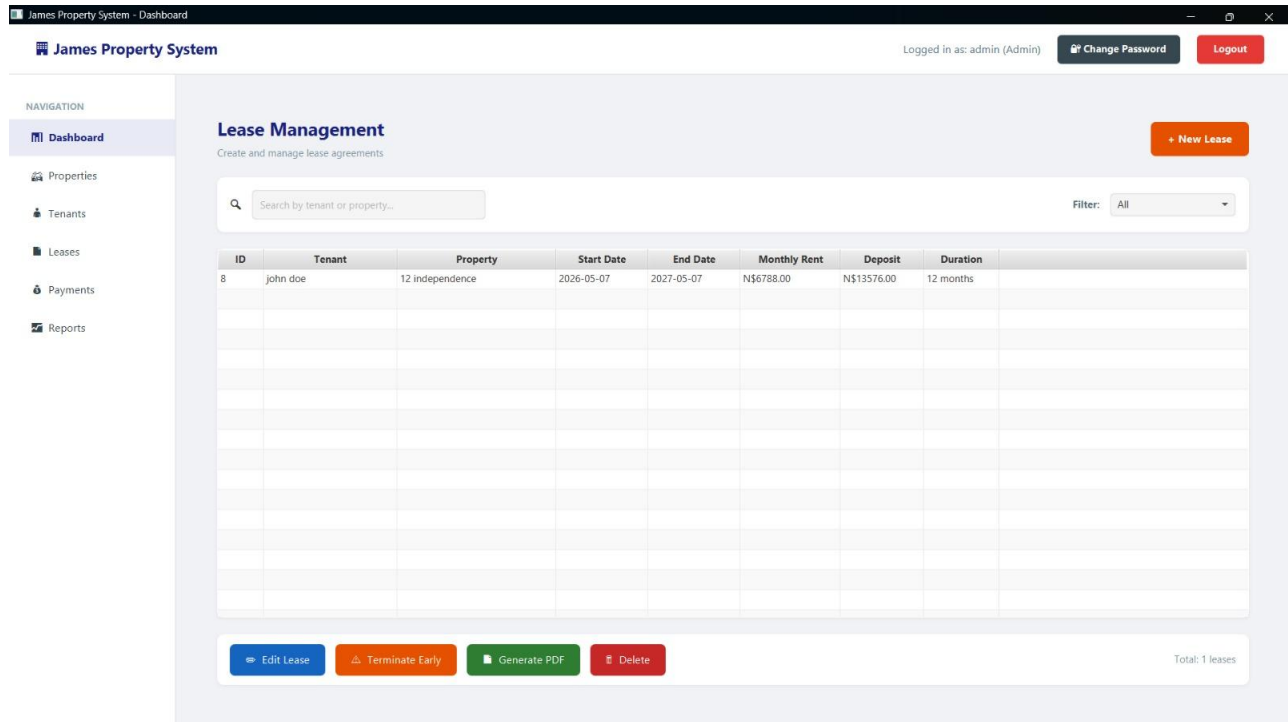
Screenshot 10:

Screenshot of Lease Management Form.

Explanation of Screenshot:

This screenshot shows the lease management section where lease agreements are created and managed.

PROGRAMMING 2 (PRG621S) GROUP ASSIGNMENT 2026



Step 8: Implementing Payment Management

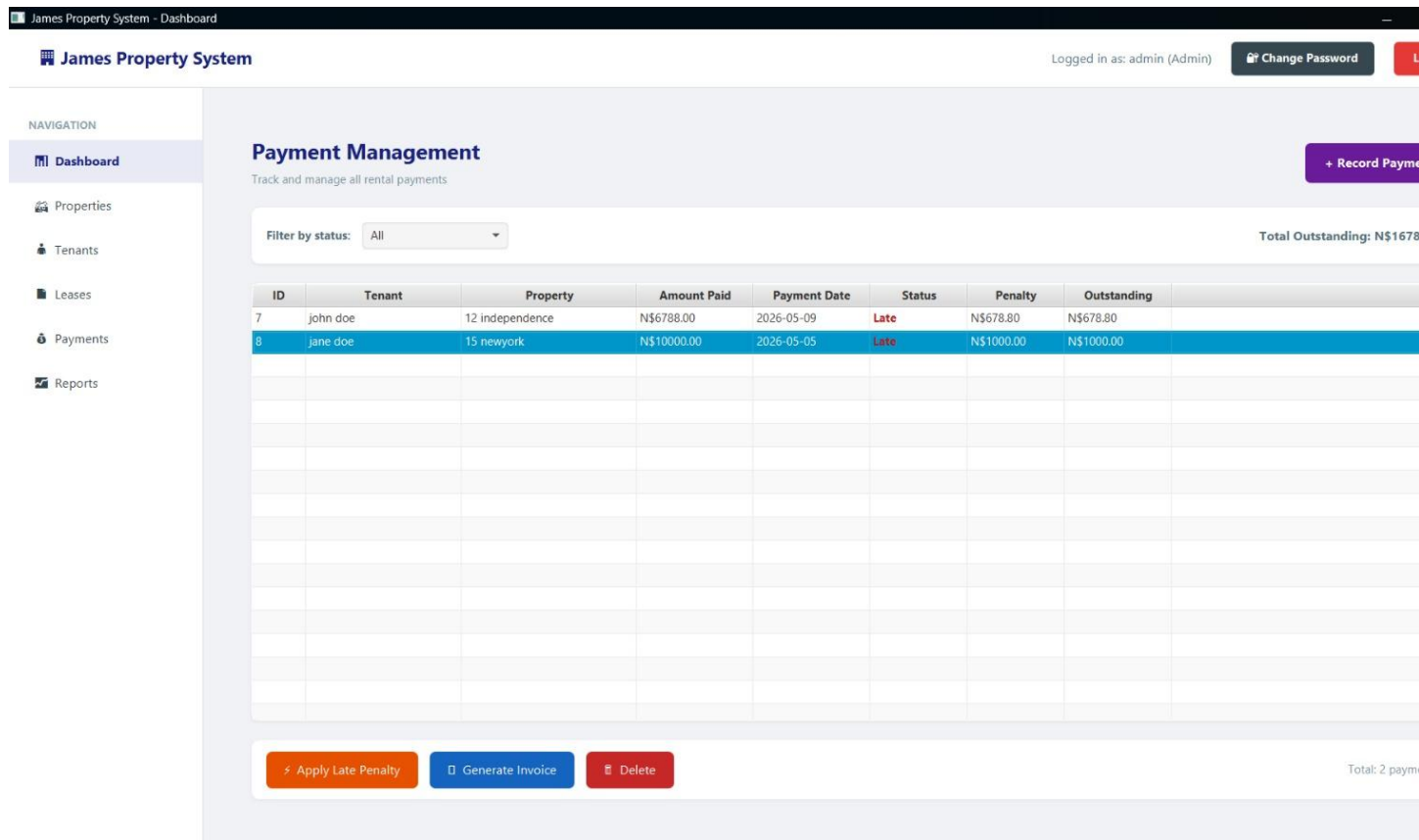
The payment module was created to record rental payments, calculate balances, and track payment status.

Screenshot 11:

Screenshot of Payment Form.

Explanation of Screenshot:

This screenshot shows the payment management interface used to record tenant payments and monitor payment status.



Step 9: Implementing Reports and File Export

The reporting module was developed to generate reports for properties, tenants, leases, and payments. Reports could also be exported to CSV or PDF format.

Screenshot 12:

Screenshot of generated report OR exported CSV/PDF file.

Explanation of Screenshot:

This screenshot shows a generated report from the system which can be viewed or exported for record keeping purposes.

PROGRAMMING 2 (PRG621S) GROUP ASSIGNMENT 2026

The screenshot shows the 'James Property System - Dashboard' interface. The user is logged in as 'admin (Admin)'. The dashboard features a navigation sidebar with 'Dashboard', 'Properties', 'Tenants', 'Leases', 'Payments', and 'Reports'. The main content area is titled 'Reports & Analytics' and includes a 'Generate and export detailed reports' section. A modal dialog box is displayed, indicating a successful PDF export: 'Report exported successfully! Saved as: Rental_Status_Report_2026-05-07.pdf'. The dashboard also shows a 'Total Properties' count of 3 and a 'This Month Revenue' of N\$16788.00. A table below the modal lists properties with columns for ID, Type, Address, and Status.

ID	Type	Address	Status
8	House	12 independence	Occupied
12	FlatApartment	easter ave	Maintenance
11	Townhouse	15 newyork	Occupied

PDF exported: Rental_Status_Report_2026-05-07.pdf

The screenshot shows the 'James Property System - Dashboard' interface, similar to the previous one, but with a different modal dialog box indicating a successful CSV export: 'CSV exported successfully! Saved as: Report_All_Properties_2026-05-07.csv'. The dashboard also shows a 'Total Properties' count of 3 and a 'This Month Revenue' of N\$16788.00. The table below the modal lists properties with columns for ID, Type, Address, and Status.

ID	Type	Address	Status
8	House	12 independence	Occupied
12	FlatApartment	easter ave	Maintenance
11	Townhouse	15 newyork	Occupied

CSV exported: Report_All_Properties_2026-05-07.csv

Step 10: Testing the System

The system was tested to ensure all modules worked correctly. Different inputs were tested to validate data entry and exception handling.

Screenshot 13:

screenshot of:

- Successful database records
OR
- JTable displaying stored data

Explanation of Screenshot:

This screenshot shows that the data entered into the system was successfully saved and retrieved from the database.

The screenshot displays the 'James Property System' dashboard. The top navigation bar includes the system name, a user login status ('Logged in as: admin (Admin)'), and buttons for 'Change Password' and 'Logout'. The left sidebar contains a 'NAVIGATION' menu with options: 'Dashboard', 'Properties', 'Tenants', 'Leases', 'Payments', and 'Reports'. The main content area is titled 'Reports & Analytics' and includes a sub-header 'Generate and export detailed reports'. It features three summary cards: 'Total Properties' (3), 'Leases' (2), and 'This Month Revenue' (N\$16788.00). A 'Report Type' dropdown is set to 'All Properties'. A modal window in the center displays a success message: 'CSV exported successfully! Saved as: Report_All_Properties_2026-05-07.csv'. Below the modal is a table with the following data:

ID	Type	Address	Status
8	House	12 independence	Occupied
12	FlatApartment	easter ave	Maintenance
11	Townhouse	15 newyork	Occupied

At the bottom of the dashboard, a status message reads: 'CSV exported: Report_All_Properties_2026-05-07.csv'.

3. Lab Challenges/Problems Encountered

- Had problems connecting the PostgreSQL JDBC driver at first.

- Database connection failed until the MySQL Connector JAR file was added correctly.
- GUI design was difficult in the beginning, therefore a code-based Swing interface was used.
- Linking database tables using foreign keys was confusing at first.
- Some SQL queries produced errors because of incorrect column names.
- Handling date formats between Java and PostgreSQL was also challenging.
- Validation was needed to prevent users from entering invalid data.
- Some buttons and forms did not refresh properly until methods were corrected.

4. Conclusion

In this project, we successfully developed a Rental Management System using Java and PostgreSQL. The system was designed to help manage property rentals in a more organized and efficient way compared to manual methods like spreadsheets and paper records. It allows users to manage properties, tenants, leases, and payments through a single application.

During the development process, we applied object-oriented programming concepts such as classes, objects, inheritance, abstraction, encapsulation, and polymorphism. Each part of the system was separated into different classes, which made the project easier to understand, maintain, and improve. We also used a service layer to handle database operations, which helped keep the code structured and clean.

The system is connected to a PostgreSQL database using JDBC, which allows all data entered through the interface to be stored permanently. This means information is not lost when the program is closed. The graphical user interface makes the system easy to use, and the dashboard provides quick access to all main features.

We also implemented exception handling and input validation to improve system reliability and prevent errors during data entry. In addition, the reporting feature allows data to be generated and exported, making it easier to view important information such as properties, tenants, leases, and payments.

Overall, this project helped us gain a better understanding of how real-world software systems are developed. It improved our skills in Java programming, database management, object-oriented design, teamwork, and GUI development. The final system meets the requirements of the case study and demonstrates how technology can be used to simplify and improve property rental management processes.

References

Oracle. (2026). *Java Documentation (Java SE)*. Available at: <https://docs.oracle.com/en/java/>
(Used for understanding Java syntax, OOP concepts, and Swing GUI components)

Oracle. (2026). *Java Swing Tutorial*. Available at:
<https://docs.oracle.com/javase/tutorial/uiswing/>
(Used for designing and implementing the graphical user interface)

MySQL. (2026). *MySQL Documentation*. Available at: <https://dev.mysql.com/doc/>
(Used for database design, SQL queries, and table creation)

W3Schools. (2026). *Java Programming Tutorial*. Available at: <https://www.w3schools.com/java/>
(Used for revision of Java concepts such as classes, inheritance, and exception handling)

GeeksforGeeks. (2026). *JDBC Tutorial in Java*. Available at:
<https://www.geeksforgeeks.org/jdbc-driver/>
(Used for understanding how to connect Java applications to MySQL databases)

NetBeans IDE. (2026). *Java Development Environment Documentation*. Available at:
<https://netbeans.apache.org/>
(Used for building and testing the Java application)